Although you might write an application entirely with D3 and your own custom code, for large-scale sustainable projects you'll have to integrate more external libraries. We'll only use one of those, colorbrewer.js, which isn't intimidating. The colorbrewer library is a set of arrays of colors, which are useful in information visualization and mapping. You'll see this library in action in section 3.3.2.

3.2 Interactive style and DOM

Creating interactive information visualization is necessary for your users to deal with large and complex datasets. And the key to building interactivity into your D3 projects is the use of events, which define behaviors based on user activity. After you learn how to make your elements interactive, you'll need to understand D3 transitions, which allow you to animate the change from one color or size to another. With that in place, you'll turn to learning how to make changes to an element's position in the DOM so that you can draw your graphics properly. Finally, we'll look more closely at color, which you'll use often in response to user interaction.

3.2.1 Events

To get started, let's update our visualization to add buttons that change the appearance of our graphics to correspond with different data. We could handcode the buttons in HTML and tie them to functions as in traditional web development, but we can also use D3 to discover and examine the attributes in the data and create buttons dynamically. This has the added benefit of scaling to the data, so that if we add more attributes to our dataset, then this function automatically creates the necessary buttons.

```
var dataKeys = d3.keys(incomingData[0]).filter(function(el) {
              return el != "team" && el != "region";
      });

d3.select("#controls").selectAll("button.teams")
             .data(dataKeys).enter()
             .append("button")
             .on("click", buttonClick)
.html(function(d) {return d;});

function buttonClick(datapoint) {
      var maxValue = d3.max(incomingData, function(d) {
              return parseFloat(d[datapoint]);
      });

var radiusScale = d3.scale.linear()
      .domain([ 0, maxValue ]).range([ 2, 20 ]);

d3.selectAll("g.overallG").select("circle")
             .attr("r", function(d) {
                    return radiusScale(d[datapoint]);
             });
};
```

Remember that dataKeys consists of an array of attribute names, so the *d* corresponds to one of those names and makes a good button title.

Builds buttons based on the data that's numerical, so we want all the attributes except the team and region attributes, which store strings

Registers an onclick behavior for each button, with a wrapper that gives access to the data that was bound to it when it was created

The function each button is calling on click, with the bound data sent automatically as the first argument